

E2E Systems Security Lab: Filesystem

In our first real lab, we will practice Rust by implementing a multi-backend simple filesystem.

Modeling an in-memory filesystem

A filesystem is a tree of directories and files. A **node** is either a file (which has some contents) or a directory (which has a list of nodes that it contains).

Notice that this definition is recursive: a node can be a directory, which contains nodes itself.

Step one

First, implement an in-memory filesystem. We said

A **node** is either a file (which has some contents) or a directory (which has a list of nodes that it contains).

Given the definition for `Filesystem`:

```
struct Filesystem {  
    root_dir: BTreeMap<String, Node>,  
}
```

complete the definition for `Node`. We want you to come up with the design for the type from the prose description. However, for additional hints, you may see the *usage* of the type that we expect from the `tests.rs` file. Please try to write the type before looking for hints, though.

Write the following function `Filesystem::print_tree`:

```
fn print_tree(&self);
```

An example output of this will be, where `/` indicates the root directory of the filesystem:

```
/
|- Cargo.toml
|- Cargo.lock
|- src/
|  |- main.rs
|- target/
|  |- debug/
|     |- twizzler
|  |- release/
|     |- twizzler
```

Note: observe that we print these in alphabetical order. It is your job to replicate this. The `BTreeMap` data structure in the standard library

`std::collections` may be useful to do so - change your directory definition if necessary.

This is not the most idiomatic way to do this. We want to be able to print the filesystem with

```
println!("{}", fs);
```

To do this, we need to implement the `Display` trait:

```
use std::fs::{self, Display};

impl Display for Filesystem {
    fn fmt(&self, f: &mut fmt::Formatter<'_,>) -> fmt::Result {
        // replace all calls to println!(...) with writeln!(f, ...)?;
    }
}
```

Step two

Implement *file handles*. A file handle is an abstraction of a reference to a file - it is the interface by which we will write and read files. This is called a file descriptor on Unix systems - it is a real concept in operating systems. In our in-memory file system, we will represent a (read-write) file handle as:

```
struct FileHandle<'a> {
    /* your job */
}
```

The `'a` lifetime means that we are *borrowing*, in some sense, from the greater file system object - you should **not** store a copy of the data in the file handle, and in fact this will not work for the write function - a write to the file handle should propagate the the node in the filesystem.

After defining this structure, then implement the following methods:

```
impl<'a> FileHandle<'a> {
    fn read(&self) -> &str;
    fn write(&mut self, data: String);
}
```

Step three

We introduce the recursive data structure `Path` that represents a path to a single file or directory:

```
enum Path {
    File(String),
    Directory(String, Box<Path>),
}
```

If the term is familiar to you, this is a linked list. This models the notion of path we have been using: the value¹

```
Directory("target", Directory("release", File("twizzler")))
```

corresponds to the path `target/release/twizzler` which means “the file `twizzler` in the `release` directory, which itself is inside the `target` directory”

Recursion tips

Recursive tasks feel much easier if you give yourself a small helper (auxiliary) method. The idea is to keep the public function clean, and move the recursion into a private helper with extra parameters.

Pattern to use:

Use a public method like `print_tree()` or `fmt(...)` that calls a helper like `print_tree_inner(level)`.

The helper takes the extra information needed for recursion (like an indent level or a current directory), and the public method stays simple.

Here are examples of **where** this helps in this lab:

- `Display for Filesystem`: have `fmt` call a function like `fmt_l(level)` on directories.
- `find`: have `find` call a helper that takes the current directory and the remaining `Path`.
- `Path::from_iter`: build the `Path` in a helper that consumes the iterator.

Recursive types in Rust

You may note that a recursive type (a type that references itself in its definition) like²

```
struct Directory {  
    name: String,  
    subdirs: Vec<Directory>,  
}
```

compiles fine, but the simpler `Path` type

```
struct Path {  
    fragment: String,
```

¹Here I am using `&str` string literals `"abc"` for convenience, but they are actually owned strings `"abc".to_owned()`

²this is a simplified version, this will not work in your code

```
    tail: Path
}
```

does **not** compile. This is because the Rust compiler needs to know the **stack size** of every type—how many bytes it takes to store a value of this type *on the stack*. For the Path example: what would the size be?

```
size(Path) = size(String) + size(Path)
           = 24 + size(Path)
```

Due to a well-known property of the natural numbers, there is no solution for the size of this type. So why does the directory example using Vec work?

```
size(Directory) = size(String) + size(Vec<Directory>)
               = 24 + 24
```

why is this the case? We look at the definition of Vec<T>:

```
struct Vec<T> {
    data: *mut T,
    size: usize,
    capacity: usize,
}
```

*mut T is a *raw pointer* of type T, and importantly (on our systems) it is just a number; a memory address. The actual data T is on the *heap*, so the compiler doesn't need to know how big the data is to calculate the size occupied on the stack. A common pattern, then, to solve the path problem we can put the inner Directory on the heap, using Rust's Box type:

```
struct Path {
    fragment: String,
    subdir: Box<Path>
}
```

which would have size

```
size(Path) = size(String) + size(Box<Path>)
           = 24 + 8
```

which is a perfectly fine fixed size. This is why the recursive case of the enum Path given above looks the way it does.

—

Your job is to implement the function:

```
impl Filesystem {
    fn find(&'a mut self, path: &Path) -> Result<FileHandle<'a>, FsError>;
}
```

Also, you will have to fill out the `FsError`³ type used in the type signature above with errors you encounter when implementing this function:

```
enum FsError {  
    /* your job */  
}
```

It will also be helpful for testing to have functions

```
impl Display for Path {  
    // your job, like you did for FileSystem  
}  
  
impl Path {  
    fn from_vec(v: Vec<String>) -> Option<Self> {  
        let mut tail = None;  
        for s in v.into_iter().rev() {  
            if let Some(t) = tail {  
                tail = Path::Directory(s, t);  
            } else {  
                tail = Path::File(s)  
            }  
        }  
  
        // we can assume that v is nonempty.  
        tail.unwrap()  
    }  
}
```

However, the second function is not idiomatic in Rust in several ways. For one, it is usually preferred to write this using iterators. Second, there is no reason for this to be a `Vec` specifically - it just has to be some sequence (that is, iterator) of values.

The way to do it in Rust is to implement the `FromIterator` trait:

```
trait FromIterator<A>: Sized {  
    fn from_iter<T>(iter: T) -> Self  
    where T: IntoIterator<Item = A>;  
}
```

which is syntactically complicated, but it just means that you can build an instance of `Path` from any iterator of the right type. You can use the following starter code:

```
impl FromIterator<String> for Path {  
    fn from_iter<T>(iter: T) -> Path
```

³It is actually better practice to call this `Error` and not `FsError`, but I wanted to be clear that this is your job to implement a filesystem-specific error type.

```

    where T: IntoIterator<Item = String>
{
    // implement here
    /* Challenge: use iterator methods on T.into_iter() to create a
Path object from an iterator of strings */
}
}

```

To use this, we use the `collect` method on iterators:

```

let v = vec!["target", "debug", "twizzler"];
let path: Path = v.into_iter().map(|s| s.to_string()).collect();
println!("{}", path); // target/debug/twizzler

```

Step four

Now, define the types:

```

struct DirectoryHandle<'a> {
    /* only you can fill in the definition */
}

```

and introduce the generalized node handle type

```

enum NodeHandle<'a> {
    /* what goes here? */
}

```

Then, generalize your `find` function to return a `NodeHandle<'a>` instead of a file handle, and implement the following functions:

```

impl<'a> NodeHandle<'a> {
    fn file_handle(self) -> Result<FileHandle<'a>, FsError>;
    fn dir_handle(self) -> Result<DirectoryHandle<'a>, FsError>;
}

```

Step five

Implement a `create` function on `DirectoryHandle` that creates a new file with a given file name, and returns a `FileHandle` pointing to it, or an error if something goes wrong (think about what the hard case for creating a file would be), and then implement an analogous function `create_dir` that instead returns a `DirectoryHandle`.

Step six

The root directory is not the only place we can find a path from. If we are “in” the `target` directory, we want to be able to find the `release/twizzler` file using a *relative* path:

```

/
|- ...

```

```
| - target/  
  | - debug/  
    | - twizzler  
  | - release/  
    | - twizzler
```

we want to be able to call `dir_handle.find(path)` just like `fs.find(path)`. Note that these have the same signature. This is a good opportunity for a trait.

Implement the

```
trait Find {  
    /* your job! */  
}
```

and implement it for `DirectoryHandle` as well as `Filesystem`, using your existing code for `Filesystem`.

Abstracting over filesystem type.

The Rust standard library proves the following functions for working with actual files on disk:

TODO max write this